GLUEBALL: AGENTIC GENERATION AND VERIFICATION OF HARDWARE-OPTIMIZED GLUON KERNELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Modern GPUs provide increasingly powerful features to improve performance, yet effectively harnessing them requires reasoning about complex execution behavior. Large Language Model (LLM) agents are exceptionally well suited to exploring this optimization space through iterative code generation, with pre-defined test-suites executed on bare metal providing correctness guarantees and performance feedback. However, numerical testing alone provides limited coverage of the input and execution space, making it possible for agents to overfit to tested inputs while missing correctness violations arising from subtle hardware-execution semantics. To address these limitations, we present **Glueball**, the first system to equip LLM agents with theorem-proving tools to reason about correctness during GPU kernel generation and optimization. Glueball targets Gluon, a tile-level Domain Specific Language (DSL) within the Triton ecosystem that provides direct, fine-grained access to GPU hardware beyond the abstractions exposed by Triton's higher-level programming model. Our system coordinates optimization across single and multi-kernel workloads, using end-to-end model performance, as well as theorem-prover guidance, as optimization feedback to evolve the kernels. Our results show improved kernel bug detection, while evaluations on real-world workloads demonstrate end-to-end performance competitive with already performant Triton baselines.

1 INTRODUCTION

Automatic GPU kernel generation using LLMs has emerged as an active area of systems research, with benchmarks such as KernelBench (Ouyang et al., 2025) and TritonBench (Li et al., 2025a) providing standardized workloads for evaluating LLM-generated kernels. This growing interest is motivated by the need for optimization outpacing traditional compiler capabilities for rapidly developing hardware. However, establishing kernel correctness through empirical testing alone is insufficient, particularly for concurrency properties such as data races and memory barrier synchronization, whose behavior can depend on subtle execution semantics (Betts et al., 2015), with modern GPU architectures exasperating this issue.

Newer architectures like NVIDIA Corporation (2023) Hopper and Blackwell introduce a substantially more asynchronous execution paradigm than earlier generations of NVIDIA hardware. For example, warp specialization allows warps (collections of threads) to take specialized producer and consumer roles, overlapping background data movement through the Tensor Memory Accelerator (TMA) with computation. Realizing these gains requires coordinating asynchronous memory transfers, computation, and hardware-backed synchronization across multiple pipeline stages, creating a substantially larger space of possible execution states in which concurrency bugs, deadlocks, and performance-destroying stalls can be difficult to expose through empirical test suites alone (Soi et al., 2026).

Additionally, the most effective use of these hardware features depends strongly on the structure of each kernel: warp specialization and asynchronous data movement may require different producer-consumer decompositions, synchronization strategies, and memory layouts across kernels. This makes fixed optimization rules or autotuning difficult to apply uniformly. LLMs offer a flexible alternative, adapting expert design patterns to each kernel, exploring qualitatively different implementations, and

iteratively refining them using profiler and debugger feedback (Ouyang et al., 2025; Wei et al., 2025; Zhang et al., 2025).

However, sustained reasoning about complex run-time semantics of GPU kernels remains a limitation of modern LLM agents (Section 4.2). To address this gap, we equip agents with Z3-based formal-verification tools that act as an interactive reasoning aid, drawing on recent work (Ospanov et al., 2026) exploring interactive settings in which LLMs work with formal theorem provers. Rather than relying solely on empirical test cases, agents can use these symbolic tools to verify properties such as race freedom, bank-conflict freedom, and reduction locality, providing formal evidence for correctness and hardware-aware optimization decisions.

We target *Gluon* because it exposes the hardware-specific controls needed for modern GPU optimization while retaining structured, tile-level semantics that are amenable to formal reasoning. Gluon sits below Triton (Tillet et al., 2019) in the compilation stack (Appendix A), preserving Triton's compiler infrastructure (Lattner et al., 2021) while exposing features such as explicit memory layouts and warp specialization to the programmer. This provides enough control to optimize asynchronous hardware primitives while leaving mechanical low-level lowering to the compiler backend.

In summary, this paper makes the following contributions:

- **Interactive profiling interface:** agents access systems engineering tools such as profilers, debuggers, and compilers to optimize GPU kernels.
- **Theorem provers as tools:** LLM agents use interactive feedback from formal theorem proving tools (via our Gluon-to-Z3 library) for verifying correctness of kernels.
- **Real-world performance testing:** Focusing on optimizing entire models rather than isolated kernels (e.g., activation functions) across representative input shapes.

2 Related Work

Automated GPU Kernel Optimization. Prior work has pursued two complementary approaches to GPU kernel generation. The first trains models to propose better kernels. AutoTriton (Li et al., 2025b) and TritonRL (Woo et al., 2025) train models to generate Triton kernels. SynKer (Sun et al., 2026) builds validated training examples across multiple backends, while Dr. Kernel (Liu et al., 2026) examines how to make execution and performance rewards more reliable. This approach extends beyond Triton to direct CUDA generation in CUDA Agent (Dai et al., 2026).

A second approach, which includes Glueball, sidesteps model training and instead develops the optimization harness that guides its revisions for a particular kernel and hardware pairing. The harness determines which tests, compiler diagnostics, and hardware measurements the model sees after each attempt. Hawkeye uses hardware feedback to guide optimization across architectures (Tschand et al., 2026), while Kernel-Smith evolves a population of candidates using compilation and performance feedback (Du et al., 2026). Astra distributes planning, generation, profiling, and testing among specialized agents (Wei et al., 2025). KernelCraft explores feedback-driven generation for unfamiliar accelerator instruction sets (Nie et al., 2026).

In addition to performance feedback, the intermediate checks must also judge whether a revised kernel is correct. Tests may miss changes to memory access or synchronization, while program verification can verify these properties with formal proof. Previous work in GPUVerify (Betts et al., 2015) builds a static verifier for CUDA and OpenCL kernels that checks for data races and barrier divergence by reducing parallel execution to a sequential verification problem. In Glueball, we bring formal checks into its Gluon optimization loop through symbolic reasoning over supported Triton IR computations and targeted Z3 models for selected Gluon hardware properties. The resulting proofs inform correctness judgments, while counterexamples guide revisions alongside test results.

Benchmarks and End-to-End Evaluation. KernelBench (Ouyang et al., 2025) established a widely used protocol for evaluating generated GPU kernels. On PyTorch workloads, it considers a generation successful when it passes predefined tests for output agreement with the reference and exceeds a chosen speedup threshold. TritonBench (Li et al., 2025a) applies a similar approach to Triton kernels, including examples drawn from real repositories. TritonGym (Guan et al., 2026) extends this setting to multi-step agents, providing compilation, testing, and profiling feedback during

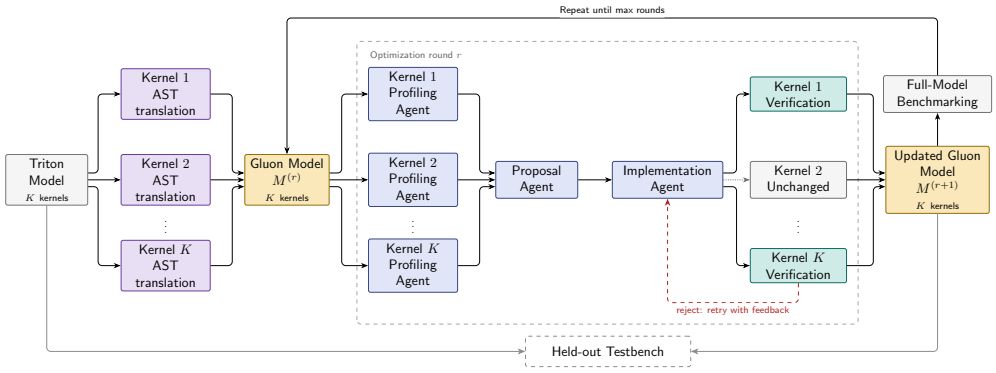


Figure 1: **System Architecture.** The optimization loop iteratively profiles, proposes, and applies kernel transformations. Every candidate variant is formally and empirically validated by the correctness pipeline prior to benchmarking and committing updates.

refinement, while retaining test-based correctness and kernel-level speed as evaluation criteria. These benchmarks make kernel-generation systems easier to compare, but their correctness results depend on the supplied tests and their performance results primarily concern isolated kernels.

RealisticTritonBench (Huang et al., 2026) extends the study beyond isolated kernel improvements by deriving tasks from real-world pull requests and reintegrating generated kernels back into those code repositories. However, static public benchmarks remain problematic: they may appear in model training data or be exploited during optimization, making measured gains difficult to generalize. Models may even exploit benchmark evaluation limitations. For example, Dr. Kernel (Liu et al., 2026) reports observing reward hacking generations that appear fast without actually running the intended Triton kernel. To address these limitations, we report results of end-to-end integration of our Gluon kernels into real-world AI models, while standalone Liger Kernels (Hsu et al., 2024) test individual optimizations against competitive Triton baselines. We introduce randomized inputs each round while keeping future cases hidden to give a useful end-to-end performance signal during optimization, and evaluate final kernels on held-out shapes and configurations.

3 Method

We jointly optimize the kernels of each model to maximize end-to-end performance. This is important for architectures such as Mixture-of-Experts (MoE) (Shazeer et al., 2017), where intertwined execution graphs mean that per-kernel improvements may not translate directly to end-to-end speedups. In particular, Amdahl's Law (Amdahl, 1967) limits the overall benefit of optimizing kernels that contribute only a small fraction of total execution time. While changes to one kernel can also interact with neighboring kernels through memory layouts, numerical precision, or synchronization. As shown in Figure 1, our system translates the reference Triton kernels to Gluon and iteratively optimizes, verifies, and benchmarks them over a fixed number of rounds, committing each round's optimizations for the next round, with each agent being equipped with a specialized toolset (Appendix B).

3.1 Initial Unoptimized Translation

Each reference Triton kernel is independently and mechanically translated into Gluon using Abstract Syntax Tree (AST) transformations that preserve functional behavior but do not optimize hardware-specific decisions. The resulting kernels serve as the initial state of the search, leaving hardware features such as TMA unused.

3.2 Hardware-Aware Optimization Pipeline

Each optimization round (Figure 1) coordinates four stages: 1) profiling, 2) proposal generation, 3) implementation and verification, and 4) benchmarking and committing the optimization.

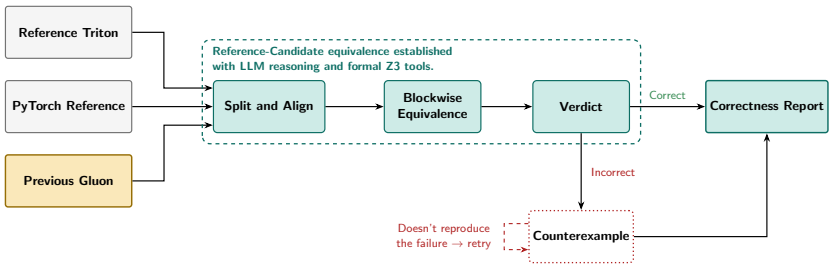


Figure 2: **Correctness Pipeline.** Reference and candidate programs are segmented and aligned into corresponding regions, which are checked for equivalence and program properties using LLM reasoning and Z3-based formal tools. Detected failures are converted into concrete witnesses and used to guide retry or produce the final correctness verdict.

Profiling. At the start of each round, an instrumented pass over the complete model measures the run-time contribution of each constituent kernel. Profiling agents use this contribution to direct their profiling effort, focusing more detailed analysis on kernels with greater impact on end-to-end performance. Agents analyze kernels in parallel using their source, historical benchmark results, and the target hardware profile. At the *coarse level*, agents use NVIDIA Nsight Compute (`ncu`) to track metrics such as SM occupancy, memory-bandwidth utilization, and register pressure. At the *fine level*, agents can inject source-level probes to isolate application-specific bottlenecks and disentangle whether a kernel is stalled on memory transfers, matrix computation, or shared-memory bank conflicts.

Proposal Generation. A centralized proposal agent combines profiling reports, historical outcomes, and the current model state to produce an ordered portfolio of candidate transformations, prioritizing transformations with the greatest expected impact on whole-model performance. A proposal may modify one or more kernels and can include multiple sequential steps. To ground its decisions, the agent can inspect Triton and Gluon intermediate representations, PTX (Parallel Thread Execution) assembly, and create and evaluate micro-benchmarks.

Implementation and Verification. The implementation agent executes the highest-ranked proposal one step at a time, applying targeted source changes to the affected kernel. Each step is checked for successful compilation, valid function signatures, and numerical correctness (both via agent-devised empirical tests and use of a Floating-Point Sanitizer (FpSan) tool), followed by **blockwise formal verification** (Section 3.3). When a check fails, the implementation agent receives the compiler diagnostics, execution tracebacks, numerical failures, and Z3 counterexamples (Appendix D). The implementation agent can then revise the change and re-run the checks. Only changes that pass all checks are committed; failed changes leave the state of kernels affected by the proposal unchanged.

Benchmarking and Committing the Optimization. After all modified kernels in a proposal have passed their per-kernel verification gates, the resulting model is executed using the current round's benchmark suite. The benchmark inputs are hidden from the agents, and the suite is expanded each round with newly randomized entries to reduce overfitting to previously observed shapes and inputs. If the integrated model fails a correctness check, the proposal is discarded; otherwise, its whole-model run time and speedup relative to the Triton baseline are recorded, and the resulting kernels become the starting point for the next optimization round.

3.3 Formal Tools-Guided LLM Kernel Verification

In an ideal world, each optimization would be formally verified to preserve program semantics, up to allowable floating-point precision differences. In practice, program equivalence is undecidable and symbolic state spaces can grow rapidly, particularly for GPU kernels orchestrating thousands of worker threads. These challenges are especially relevant to Gluon, which exposes low-level hardware primitives such as asynchronous copies, tensor-core accesses, and memory barriers.

We therefore use a hybrid verification approach combining **symbolic execution**, **LLM reasoning**, and **targeted Z3 models**. Symbolic execution provides formal reasoning for computations that can be represented directly, while the LLM reasons across semantics that fall outside this representation and uses Z3 explicitly when a hardware-level property requires a specialized model.

Symbolic Execution. Our symbolic execution model maps supported IR operations to Z3 expressions. Arithmetic operations are translated to corresponding Z3 operators, while supported mathematical operations use Z3 floating-point operators or uninterpreted functions (Appendix C). Input tensors are represented by symbolic values (instead of arrays in memory), and the interpreter follows SSA definition chains to construct symbolic expressions for derived values.

For example, applying the interpreter to the Triton softmax kernel in Appendix C.1 produces an expression of the form:

$$\text{softmax_output} = \frac{\exp(\text{load_row} - \text{maxnumf_row_minus_max})}{\text{addf_denominator}}$$

with opaque symbol names prepended to the input value they operate on. Note that this reflects the mathematical formula $\text{softmax}(x_i) = \frac{\exp(x_i - x_{\max})}{\sum_j \exp(x_j - x_{\max})}$, but was extracted from the Intermediate Representation (IR). We defer to Appendix C for a full list of supported operations and their Z3 representations.

Opaque Operations and Symbolic Regions. Operations whose internal behavior is not modeled symbolically—including reductions, memory accesses, synchronization primitives, and loop structures—are treated as *opaque operations*. Their results are represented as symbolic values whose internal derivation is left unspecified. These operations partition the kernel into *symbolic regions*: contiguous sequences of supported operations that can be translated into closed-form symbolic expressions. Z3 provides formal reasoning within each region, while the LLM reasons about dependencies across opaque boundaries.

Our Gluon-to-Z3 Library. For Gluon kernels, the LLM can additionally use our *Gluon-to-Z3 Library*, specifically designed to model hardware-level behavior that cannot be captured by value-level symbolic execution. These models allow Z3 to check properties such as asynchronous race freedom, barrier ordering, and shared-memory correctness (Appendix D).

3.4 Correctness Pipeline

Figure 2 shows how these components are combined into the correctness pipeline. We next describe each component in detail.

Reference-Candidate Equivalence. The pipeline first compares an optimized kernel against its reference PyTorch and Triton implementations. Optimizations such as tile re-blocking, warp specialization, and loop unrolling can change the structure of the candidate IR, so corresponding symbolic regions may not align directly. A *Splitter Agent* identifies symbolic regions and an *Alignment Agent* aligns their inputs and outputs. An *Equivalence Agent* then checks each aligned pair using their Z3 representations and LLM reasoning. For Gluon-specific hardware properties, the LLM instead constructs the relevant formal model using the Gluon-to-Z3 Library Appendix C and invokes Z3 to check the property Appendix D.

Counterexample Synthesis. When a formal check fails, the pipeline extracts the available evidence and constructs a concrete explanation. Equivalence failures provide the divergent symbolic results, while hardware-property failures can provide Z3 counterexamples describing violating assignments or executions, such as thread coordinates, memory addresses, or conflicting events Appendix D.

Verdict Agent. This agent combines blockwise equivalence results with the kernel source to approve the kernel or identify the failure mechanism. It can execute targeted code snippets or interactive debugger sessions to synthesize a runnable harness with the relevant inputs and launch parameters. Executing this harness on physical GPU hardware provides a concrete witness that can be used to confirm the failure and guide subsequent repair.

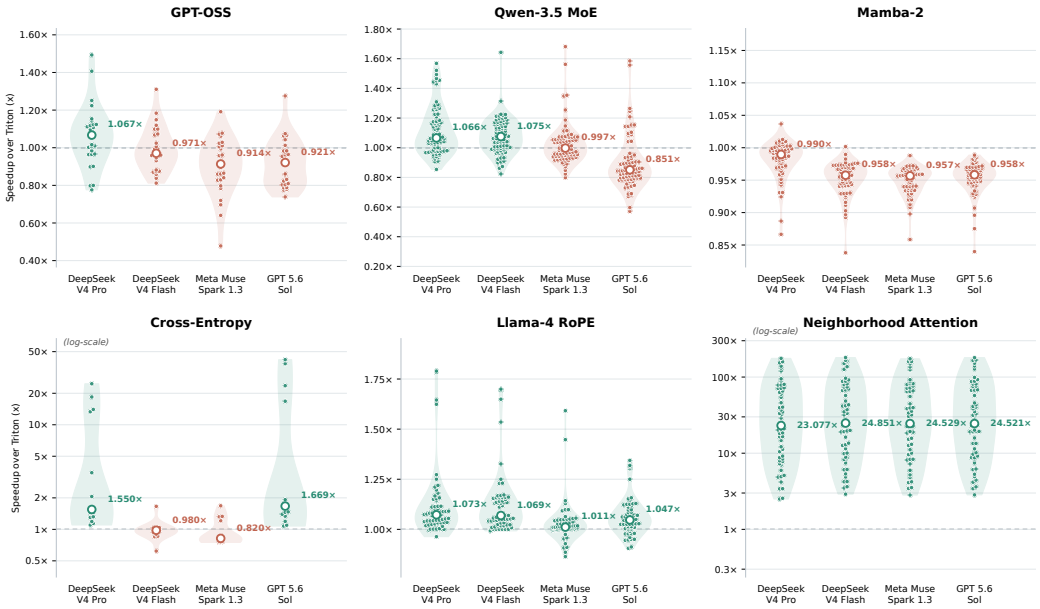


Figure 3: **Best-round speedup across input shapes.** Each violin shows the density of per-input shape speedups (models evaluated in an end-to-end configuration). The median is denoted by the circle, its color indicates if it is at or above parity (green) or below parity (red). Each point denotes an input shape configuration. Note that some panels use log-scale.

4 Experiments

We evaluate Glueball on two objectives: end-to-end performance optimization (across a budget of 6 rounds) and Gluon kernel correctness verification on an NVIDIA DGX Spark with a GB10 chip. We instantiate our pipeline with agents powered by Deepseek V4 Pro & Flash, Meta Muse Spark 1.3 and GPT 5.6 Sol models, enabling a comparison of performance across models. Complete experiment data can be found in Appendix E.

4.1 Optimization Performance Evaluation

Our evaluation consists of three model workloads (*GPT-OSS*, *Qwen-3.5 MoE* and *Mamba-2*), and three standalone kernels (*Neighborhood-Attention*, *Cross-Entropy*, and *Llama-4 RoPE* from Liger Kernels), chosen to expose complementary optimization challenges including Tensor Core utilization, asynchronous data movement, latency hiding, and shared-memory management. Our optimization goal is to minimize end-to-end model (and standalone kernel) execution time across a wide range of input shapes. Optimization performance data can be found in Figure 3 and the kernels optimized jointly for each model workload are detailed in Appendix E.1.

GPT-OSS: optimizing the data path to Tensor Cores. DeepSeek V4 Pro generates a kernel improving end-to-end performance over the Triton baseline, highlighting the importance of optimizing the data path alongside the computation. It removes several unnecessary tensor transformations from the Triton reference. The Query/Key/Value transpose is encoded in the TMA descriptor rather than performed as a physical copy and Grouped-Query Attention (GQA) is handled by computing the corresponding Key-Value head index in-kernel rather than materializing eight copies of each Key-Value head. Additionally, TMA handles out-of-bounds accesses without padding tensors and the output is written directly in its final layout rather than transposed and copied afterwards. These changes reduce memory traffic and allow multiple query heads to reuse the same underlying Key-Value data.

DeepSeek V4 Flash and Meta Muse Spark 1.3 instead focus on improving the `_attn_fwd` kernel's arithmetic path, replacing TF32 with lower-precision MMA operations, as hardware support often makes BF16 and FP16 operations faster. However, DeepSeek V4 Flash's lower precision optimization

Model	Generic Correctness Suite				Synchronization Correctness Suite			
	Detected (of 24) ↑		False Positives (of 26) ↓		Detected (of 10) ↑		False Positives (of 12) ↓	
	Baseline	Glueball (Δ)	Baseline	Glueball (Δ)	Baseline	Glueball (Δ)	Baseline	Glueball (Δ)
DeepSeek V4 Pro	21	+3	0	0	10	0	7	-5
DeepSeek V4 Flash	22	+2	0	0	10	0	1	-1
Meta Muse Spark 1.3	22	+2	0	0	9	0	3	-3
GPT 5.6 Sol	21	+2	0	0	9	+1	1	-1

Figure 4: **Correctness Test Suite results**. The Glueball correctness pipeline improves bug detection across all four models tested while maintaining fewer or equal amounts of false positives across both test suites. Results on the synchronization suite highlight the benefit of formal reasoning tools for exposing asynchronous execution errors that are difficult to infer from source code alone.

changes the MMA instruction shape necessitating explicit `convert_layout` operations, incurring substantial overhead. Interestingly, GPT 5.6 Sol chooses to optimize performance over the arithmetic path, however, it is mistakenly attempts to use `tcgen05` and Tensor Memory, which are not available on the test machine, despite having access to the hardware configuration.

Qwen-3.5 MoE: mapping expert routing to TMA. Qwen-3.5 MoE illustrates how agents can express the existing algorithm from the Triton reference more effectively on the hardware. The DeepSeek V4 Pro- and Flash-generated kernels preserve the reference algorithm and numerical formulation, but replace compiler-generated data movement with explicit Gluon implementations. The key opportunity is the irregular access pattern introduced by expert routing. While the Triton kernel gathers routed token rows using masked pointer loads, both DeepSeek V4 Pro and DeepSeek V4 Flash use TMA's gather operation to move those rows directly into shared memory. The expert-specific weight tiles, which have regular layouts, are transferred separately using contiguous TMA, matching each access pattern to a corresponding hardware data-movement mechanism. GPT 5.6 Sol optimization rounds repeatedly choose to optimize the `_fused_down_proj_kernel`, however the proposed changes it applies reduce performance. Thus, it's optimization effort is spent on recovering previous performance, with `_fused_up_proj_swiglu_kernel` remaining untouched before exhausting the total round budget.

Mamba-2: competing against an already strong compiler schedule. Mamba-2's `chunk_scan` presents a different challenge: improving on an already strong Triton software pipeline. All agents explore several valid pipeline structures, varying stage counts and buffering, eventually eliminating register spills, and substantially reducing the number of executed instructions. However, these improvements do not translate into lower run time: the resulting kernels remain slower than the Triton baselines because often its additional buffering and synchronization consumes more resources without providing sufficient operation-overlap to compensate. Finding a better schedule for an already well-scheduled kernel requires exploring a larger space of pipeline configurations, suggesting that either more optimization rounds or stronger hardware-specific priors about effective pipeline structures could be necessary.

Standalone kernels. Neighborhood attention demonstrates that the optimization search can improve the underlying algorithm, rather than only its hardware utilization. The reference Triton implementation materializes the full $L \times L$ attention matrix, whereas all optimized Gluon kernels directly traverse the local attention window, reducing the relevant work from $O(L^2D)$ to $O(LWD)$, where L is the sequence length, W is the local attention window, and D is the hidden dimension. In contrast, the improvements by DeepSeek V4 Pro and GPT 5.6 Sol on the cross-entropy experiment primarily stem from reducing host-side launch overhead rather than from a substantially faster kernel, while remaining consequential for end-to-end performance. Llama-4 RoPE has little headroom because its elementwise, bandwidth-bound reference already comes close to saturating device memory bandwidth, but we observe that all Gluon implementation can nonetheless achieve the same.

(a) Pipelining bug

```
# Correct
cp.wait_group(num_buffers - 1)
perform_add(
    a_smem[read_idx % num_buffers],
    b_smem[read_idx % num_buffers],
)

# Bug
cp.wait_group(num_buffers)
perform_add(
    a_smem[read_idx % num_buffers],
    b_smem[read_idx % num_buffers],
)
```

(b) Symbolic model

```
def build(num_buffers, wait_arg):
    ctx = GluonVerificationContext()
    r0, w0 = ctx.map_async_copy_global_to_shared(
        0, src_addr=0x1000, dst_addr=0x100,
        size=4, label='copy0')
    ctx.map_async_commit_group(0)
    r1, w1 = ctx.map_async_copy_global_to_shared(
        0, src_addr=0x2000, dst_addr=0x200,
        size=4, label='copy1')
    ctx.map_async_commit_group(0)
    wait = ctx.map_async_wait_group(
        0, num_outstanding=wait_arg,
        label=f'wait({wait_arg})')
    load = ctx.map_load(
        0, 0x100, Space.SHARED,
        size=4, label='read buffer0')
    ctx.program_order(wait.id, load.id)
    return ctx

for wait_arg in [2, 1]:
    safe, model = build(
        2, wait_arg).check_async_race_freedom()
```

(c) Z3 verdict

```
wait_group(2):
DATA RACE detected:
copy0.smem_write
  STORE SHARED@0x100
  proxy=ASYNC_COPY
<-race->
read buffer0
  LOAD SHARED@0x100
  proxy=GENERIC

wait_group(1):
RACE-FREE: proven
```

(d) Pipeline witness

```
check_pipeline_stage_isolation(
    num_stages=2, max_inflight=2)

PIPELINE COLLISION:
producer and consumer share a slot
witness: [iter_p = 2, lag = 2]
-> (False, [iter_p = 2, lag = 2])

max_inflight=1:

STAGE-ISOLATED:
num_stages=2 never aliases
with up to 1 in-flight iteration
-> (True, None)
```

Figure 5: **Theorem-prover-guided debugging of a pipelining bug.** (a) The buggy kernel waits for too few completed asynchronous copies before reading from a shared-memory buffer. (b) The LLM translates the relevant operations into a symbolic model, (c) Z3 identifies the conflicting asynchronous write and shared-memory read, and (d) a pipeline-level check produces a concrete iteration witness showing that the producer and consumer can alias the same buffer slot.

Summary. Overall, optimization effectiveness varies across workloads according to the opportunities exposed. GPT-OSS and Qwen-3.5 MoE benefit from structured data-movement optimizations that better match operand delivery to hardware through explicit TMA use, while Mamba-2 shows the difficulty of discovering improvements in the larger scheduling space of software pipelining. Neighborhood attention further demonstrates that the search can identify algorithmic opportunities, reducing computational complexity rather than merely optimizing the existing implementation.

4.2 Correctness Pipeline Evaluation

We evaluate the correctness pipeline independently of the optimization results using two benchmark suites that target complementary aspects of Gluon kernel correctness. The *Generic Correctness Suite* covers common kernel-level bugs, while the *Synchronization Correctness Suite* targets asynchronous execution and synchronization semantics. For each suite, we compare each model with and without the full correctness pipeline. We count a defect as correctly detected only when the model identifies the bug and its failure mechanism.

Generic Correctness Suite. Our suite contains 50 cases drawn from seven Liger-Kernel operators, including 24 cases with introduced bugs and 26 semantics-preserving controls. The bugs cover masking, numerical stability, accumulator precision, memory addressing, and algorithmic computa-

tion. Figure 4 shows that the correctness pipeline improves detection across all four models while maintaining zero false positives on the controls.

Synchronization Correctness Suite. The synchronization suite contains 22 Gluon kernels, including 10 buggy cases and 12 controls, targeting asynchronous copies, TMA operations, pipelining, and warp specialization. As shown in Figure 4, the pipeline both recovers missed defects and substantially reduces false positives compared to the baseline, indicating that asynchronous execution semantics are difficult to reason about for current LLMs from source code alone.

We next examine a pipelining bug observed during correctness pipeline evaluation to illustrate how the correctness pipeline turns this reasoning into explicit evidence (Figure 5). The example kernel uses two shared-memory buffers in a circular pipeline: while one buffer is consumed, the next is refilled asynchronously. The consumer must therefore wait for the copy that populates the buffer it is about to read. The buggy `wait_group(num_buffers)` leaves this copy outstanding, whereas `wait_group(num_buffers - 1)` provides the required ordering. Rather than relying on the LLM to infer asynchronous execution semantics from source code alone, Figure 5 shows how the LLM agent utilizes theorem-prover checks to expose both the low-level race and a concrete pipeline-level witness.

Summary. Overall, the correctness pipeline evaluation results show that it improves LLMs' ability to identify both kernel bugs and their underlying failure mechanisms, while reducing false positives. By combining symbolic execution with theorem-prover checks, the pipeline provides explicit evidence that enables LLMs to reason about correctness properties that are difficult to infer reliably from kernel source alone.

5 Conclusion and Future Work

We present **Glueball**, the first system to integrate formal verification tools into LLM-assisted GPU kernel optimization. Glueball enables LLM agents to iteratively optimize Gluon kernels against real hardware while using formal tools to verify the correctness of their generated kernels. Across model workloads and standalone kernels, we find that agents can improve end-to-end performance by exploiting opportunities in data movement, hardware-specific execution, and algorithmic structure. In addition, Glueball demonstrates how LLM agents can use theorem-proving tools within the optimization loop to verify their optimizations, improving bug detection and reducing false positives: agents translate relevant kernel computations into symbolic representations, invoke Z3 to check equivalence and hardware-specific properties, and use counterexamples to identify and diagnose correctness failures. Overall, our results support integrating formal verification directly into agentic GPU optimization, complementing existing approaches based on hardware feedback and iterative search with explicit verification of the resulting kernels.

Multi-GPU and heterogeneous optimization. An important future direction is to extend our agentic generation and verification methodology to distributed kernels, following the multi-GPU workloads introduced by ParallelKernelBench (Chan et al., 2026), and to heterogeneous accelerator targets studied by KernelGenBench (Zang et al., 2026). Multi-GPU execution would introduce proof obligations of collective ordering, cross-rank synchronization, communication–computation overlap, and distributed race and deadlock conditions. These extensions should be evaluated through end-to-end distributed model throughput and cross-platform robustness, rather than isolated kernel or communication latency.

Constraint and statistics-guided search. Software-pipelining schedules, including those considered for Mamba-2, expose a large combinatorial optimization space. Future systems could extend our existing configuration and safety checks to constrain parameterized schedule families before generating concrete implementations, for example, excluding configurations that violate hardware-resource constraints. Agents could use these constraints, together with accumulated profiling results and verification feedback, to refine their optimization strategies across iterations. Statistical optimization methods could further guide this process by identifying promising configurations from previous evaluations. Combining these methods with agent reasoning could enable broader exploration with fewer costly LLM iterations and hardware evaluations.

AI USE STATEMENT

AI was used to both aid and polish writing. It was also used for retrieval and discovery of related work and prior methods. Additionally, AI was used for ideation and coding agents for implementation.

REPRODUCIBILITY STATEMENT

Upon publication, we plan to release our code, benchmark suites, and experiment results in open-source.

REFERENCES

Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, volume 30, pp. 483–485, Atlantic City, NJ, 1967. AFIPS Press.

Adam Betts, Nathan Chong, Alastair F Donaldson, Jeroen Ketema, Shaz Qadeer, Paul Thomson, and John Wickerson. The design and implementation of a verification technique for gpu kernels. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 37(3):1–49, 2015.

Willy Chan, Nathan Paek, Simon Guo, Simran Arora, and Daniel Y Fu. Parallelkernelbench: Can llms write fast multi-gpu kernels? In *Deep Learning for Code: Towards Human-Centered Coding Agents*, 2026.

Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, et al. Cuda agent: Large-scale agentic rl for high-performance cuda kernel generation. *arXiv preprint arXiv:2602.24286*, 2026.

He Du, Qiming Ge, Jiakai Hu, Aijun Yang, Zheng Cai, Zixian Huang, Sheng Yuan, Qinxiu Cheng, Xinchen Xie, Yicheng Chen, et al. Kernel-smith: A unified recipe for evolutionary kernel optimization. *arXiv preprint arXiv:2603.28342*, 2026.

Yue Guan, Yichen Lin, Xu Zhao, Jianzhu Yao, Xinwei Qiang, Zhongkai Yu, Pramod Viswanath, Yufei Ding, and Adnan Aziz. Tritongym: A benchmark for agentic llm workflows in triton gpu code generation. In *Forty-third International Conference on Machine Learning*, 2026.

Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*, 2024.

Jinjun Huang, Zhongzhen Wen, Tongtong Xu, Meng Yan, Xin Xia, and Zhongxin Liu. Realistictritonbench: A benchmark for triton-kernel generation in real-world ai frameworks. *arXiv preprint arXiv:2608.12004*, 2026.

Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM international symposium on code generation and optimization (CGO)*, pp. 2–14. IEEE, 2021.

Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, WangHaojie WangHaojie, Jianrong Wang, Xu Han, et al. Tritonbench: Benchmarking large language model capabilities for generating triton operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 23053–23066, 2025a.

Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, et al. Autotriton: Automatic triton programming with reinforcement learning in llms. *arXiv preprint arXiv:2507.05687*, 2025b.

Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. Dr. kernel: Reinforcement learning done right for triton kernel generations. *arXiv preprint arXiv:2602.05885*, 2026.

Jiayi Nie, Haoran Wu, Yao Lai, Zeyu Cao, Cheng Zhang, Binglei Lou, Erwei Wang, Jianyi Cheng, Timothy M Jones, Robert Mullins, et al. Kernelcraft: Benchmarking for agentic close-to-metal kernel generation on emerging hardware. *arXiv preprint arXiv:2603.08721*, 2026.

NVIDIA Corporation. NVIDIA H100 Tensor Core GPU Architecture. White paper, NVIDIA Corporation, 2023. URL https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c.

Azim Ospanov, Zijin Feng, Jiacheng Sun, Haoli Bai, Xin Shen, and Farzan Farnia. Hermes: Towards efficient and verifiable mathematical reasoning in llms, 2026. URL https://arxiv.org/abs/2511.18760.

Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*, 2025.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017. URL https://arxiv.org/abs/1701.06538.

Rupanshu Soi, Rohan Yadav, Fredrik Kjolstad, Alex Aiken, Maryam Mehri Dehnavi, Michael Garland, and Michael Bauer. Optimal software pipelining and warp specialization for tensor core {GPUs}. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pp. 1875–1890, 2026.

Qi Sun, Robert Tjarko Lange, and Alex L Zhang. Synker: Synthesize, kernelize, reinforce-teaching gpu kernel generation to small language models. In *Deep Learning for Code: Towards Human-Centered Coding Agents*, 2026.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.

Arya Tschand, Kesavan Ramakrishnan, Alexander Ingare, Simon Guo, Jeffrey Jian Ma, Zishen Wan, Simran Arora, Azalia Mirhoseini, and Vijay Janapa Reddi. Hawkeye: Hardware-aware gpu kernel optimization with minimal supervision. In *Deep Learning for Code: Towards Human-Centered Coding Agents*, 2026.

Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. Astra: A multi-agent system for gpu kernel performance optimization. *arXiv preprint arXiv:2509.07506*, 2025.

Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. Tritonrl: Training llms to think and code triton without cheating. *arXiv preprint arXiv:2510.17891*, 2025.

Peiyu Zang, Jian Tao, Jialing Zhang, Yichen Yuan, Wentao Zhang, Guang Liu, and Yonghua Lin. Kernelgenbench: Can llms and agents write efficient kernels across operator sources and hardware platforms?, 2026. URL https://arxiv.org/abs/2607.27231.

Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. Cudaforge: An agent framework with hardware feedback for cuda kernel optimization. *arXiv preprint arXiv:2511.01884*, 2025.

A TRITON AND GLUON COMPILATION PIPELINES

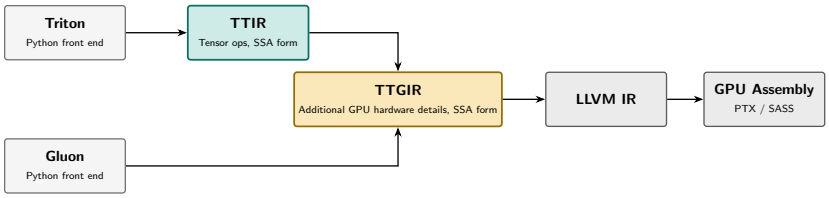


Figure 6: Triton and Gluon compilation pipelines.

Both Triton and Gluon lower Python kernels through Triton's intermediate representations to GPU assembly (Figure 6). Triton first lowers the Python program to TTIR (Triton Intermediate Representation), which represents tensor operations in SSA (Static Single Assignment) form, and then to TTGIR (Triton GPU Intermediate Representation), which introduces additional GPU hardware details before lowering through LLVM IR to GPU assembly.

Gluon provides a Python front-end directly for TTGIR, allowing hardware-specific details to be expressed explicitly while reusing Triton's downstream compilation pipeline. Compared to GPU programming languages such as CUDA, Gluon retains Triton's blockwise programming model.

B TOOL AND CLI UTILITY INVENTORY

Every agent in Figure 1 is equipped with a role-specific bundle of toolsets (Table 1). We distinguish between toolsets specifically designed for this work (Appendix B.1) and general command-line programs and packages used during kernel development (Appendix B.2) by a systems/kernel engineer. All tools are isolated with `bubblewrap`, providing a read-only view of the filesystem outside the agent's own workspace and no network access.

Agent role(s)	Capabilities
Gluon translation	**IR** **Z3** **Doc** **Gluon seeding**
Gen.-benchmark, Gen.-launch	**Bench** **Debug** **NCU**
Profiling	**Bench** **Debug** **NCU** **Proton**
Proposal, Optimization	**IR** **Z3** **Bench** **Doc** **Debug** **NCU** **Proton** **Gluon seeding**
Splitter, Mapper	**Z3** **Doc** **Debug**
Equivalence, Verdict	**IR** **Z3** **Doc** **Debug** **Gluon seeding**

Table 1: **Capability categories bound into each agent role's tool bundle.** **IR** = Triton/TTIR/TTGIR extraction; **Z3** = symbolic execution and formal program property verification; **Bench** = self-generated-harness creation/execution; **Doc** = scoped documentation search; **Debug**/**NCU**/**Proton** = CLI tool wrappers detailed in Appendix B.2.

B.1 SPECIALIZED TOOLS

The tools below allow agents to perform their specialized roles and go beyond what is achievable via command line tools alone.

Tool	Description
IR extraction and compilation	
`get_triton_ir`	Compile a Triton kernel and return its TTIR and TTGIR lowering.
`get_gluon_ir`	Compile a Gluon kernel and return its TTGIR lowering.

Tool	Description
`pytorch_to_compiled_triton`	Capture the Triton kernel generated by PyTorch's TorchInductor backend for a `torch.compile`'d function, when no hand-written reference Triton kernel is available.
Symbolic execution and formal verification (Z3)	
`symbolize_triton`	Compile and execute a Triton block, extracting the Z3 symbolic representation (plus execution output) (Appendix C.1).
`symbolize_ttir`	Symbolically execute already-obtained TTIR into Z3 symbolic formulas, without recompiling.
`run_z3`	Execute arbitrary Python Z3 code; auto-imports the project's Gluon-to-Z3 library (Appendix D).
Benchmarking	
`prepare_benchmark`	Assemble a kernel and the kernel's launch code (contains auto-tuning code and Gluon 'constexprs') (Section 3.1) into a runnable benchmark script.
`test_benchmark`	Run the auto-benchmark harness end-to-end, on every configured input, to validate launch code before finalizing it.
Documentation	
`read_doc_file`	Read one file from the project's reference documentation tree.
`query_documentation`	Delegate an open-ended documentation question to a dedicated documentation sub-agent.
Gluon seeding	
`translate_triton_source`	Mechanically translate a `@triton.jit` kernel into an equivalent, unoptimized `@gluon.jit` kernel via an AST-based translator, seeding every run's search; also invocable during later rounds, as a reference for correct Gluon API usage.

Table 2: Description of the tools making up the non-CLI toolsets.

B.2 General command line tools and wrappers

In addition to the tools above, every agent has access to a general-purpose shell (via `run_command`/`start_command`/ `check_command`/`stop_command`). This gives every agent standard text-processing tools (`grep`/`ripgrep`, `sed`, `awk`, arbitrary pipes), file read/write, and the ability to directly invoke `ncu`, `pdb`, `proton-viewer`, and arbitrary Python scripts. For the three capabilities below, rather than relying on an agent to assemble the right command invocation from scratch every time, we additionally provide utilities that help build scripts or commands an experienced engineer would use.

Capability	Description
Debug (interactive)	An interactive, multi-turn `pdb` session over a persistent subprocess, for a kernel. It allows for stepping through execution and inspecting intermediate values. A one-shot fallback (assemble a script and run it once, rather than stepping through interactively) is also available for quick comparisons.
`ncu` profiling	Assembles a validated NVIDIA Nsight Compute runner script and prints the exact `ncu` shell command to run it; also helps perform a differential, metric-by-metric comparison of two saved profiles from previous `ncu` runs.
Proton profiling	Assembles a script instrumented with Triton's own Proton profiler, at either coarse inter-kernel timing or cycle-accurate intra-kernel granularity, and provides a SQL-query interface over the resulting timeline trace.

Table 3: CLI-wrapped capabilities: wrappers around external programs (`pdb`, `ncu`, Proton) that already exist independently of this project.

C MLIR DIALECTS AND THEIR Z3 REPRESENTATION

The Z3 model of TTIR (Section 3.3) dispatches MLIR operations by dialect prefix to five handler modules; the same interpreter also processes TTGIR through the `ttg` handler. We use this interpreter to divide kernels into *symbolic regions*: contiguous regions of IR for which all operations admit an explicit symbolic interpretation. Operations that are not modeled by the interpreter are treated as *opaque operations*. Their results are represented by fresh symbolic values, which we refer to as *opaque inputs* when they serve as inputs to subsequent symbolic regions.

Every tensor value is modeled by a single representative scalar element, with no array or index theory involved. Consequently, the interpreter provides value-level symbolic reasoning rather than a complete model of tensor shapes, layouts, or memory behavior; such properties are handled indirectly through address arithmetic, explicit Z3 queries, or the Gluon-to-Z3 Library (Appendix D). Since TTIR and TTGIR use Static Single Assignment form, the formula for a supported value is obtained by walking its chain of definitions back to kernel arguments and opaque inputs. When an operation is not supported by the interpreter, the model does not guess its semantics; instead, it collapses the result to a fresh symbol.

The five handlers differ in how much semantic information they expose to the symbolic model. Arithmetic and supported mathematical operations contribute directly to symbolic expressions; tensor reshaping and pointer arithmetic can often be represented without expanding tensor state; and memory, reduction, collective, and control-flow operations are partially modeled or treated as opaque. Table 4 summarizes these behaviors and the resulting boundaries of symbolic regions. To make these distinctions concrete, Appendix C.1 works through a real kernel end-to-end, illustrating both how supported operations are translated into symbolic expressions and where the interpreter reaches its limits.

Table 4: MLIR dialects handled by the Z3 model of TTIR/TTGIR, the number of operations each handler dispatches on, and how each dialect is represented. An operation whose dialect prefix is not in this table is always collapsed to a fresh opaque symbol.

Dialect	**Ops**	**Representative ops**	**How it's modeled**
`arith`	51	`addf`, `cmpi`, `select`, `sitofp`, `bitcast`	Every operation maps directly to a native Z3 bit-vector or floating-point operator (e.g. `addf`→`fpAdd`, `divui`→`UDiv`)—an exact translation with no opaque terminals.
`math`	46	`exp`, `log`, `sin`, `sqrt`, `erf`, `ctlz`	Split by how well-behaved the operation is: numerically simple ops (`sqrt`, rounding, `fma`, `abs`, NaN/Inf tests) map to native Z3 floating-point builtins and are evaluated exactly; transcendentals (`exp`, `log`, trigonometric, `pow`) become *uninterpreted* Z3 functions—not numerically evaluated, but provably equal whenever applied to syntactically identical arguments, which suffices for elementwise equivalence checks.
`tt` (Triton core)	52	`load`/`store`, `addptr`, `splat`/`reshape` /`trans`, `reduce`, `dot`, `atomic_rmw`	A three-way split: pure reshaping/aliasing ops (`splat`, `broadcast`, `reshape`, `trans`, …) re-tag the same expression at zero cost; pointer arithmetic is concrete bit-vector math; and every memory access or collective op—`load`, `reduce`/`scan`, `dot`, atomics, `gather` – becomes a fresh opaque symbol, since what it actually computes is never modeled.

continued on next page

Dialect	Ops	Representative ops	How it's modeled
`scf`	12	`for`, `if`, `while`, `index_switch`	Approximated, not modeled exactly: `if` is exact (both branches interpreted and merged with Z3's own `If`); `for`/`forall`/ `parallel` run their body *once* symbolically, so loop-carried values become fresh symbols after that single pass rather than being unrolled; `while` is always opaque, since general loop termination is undecidable.
`ttg` (Triton GPU)	27	`local_load`/`store`, `convert_layout`, `async_copy_global_to_local`, `warp_specialize`	Lets the same interpreter process TTGIR, the dialect Gluon compiles to directly (Section 3.3): layout-conversion and reshape ops are aliasing, local-memory reads are opaque and writes are recorded as side effects, and only the *default* partition of `warp_specialize` is traced, since the other partitions run in parallel on different hardware and fall outside a single-threaded interpreter's model.

C.1 WORKED EXAMPLE: `SOFTMAX`

To make the symbolic execution mechanism concrete, this section runs the row-wise `softmax` kernel from Triton's tutorials through compilation to TTIR and then through the Z3 interpreter, via the `get_triton_ir` and `symbolize_ttir` tools (Appendix B) respectively.

Each kernel invocation loops over a slice of rows, and for every row: loads it, subtracts the row's max for numerical stability, exponentiates, normalizes by the row's sum, and writes the result back. Compiling one loop iteration to TTIR (location metadata and pointer-address bookkeeping elided for readability) gives:

```
scf.for %row_idx = %row_start to %n_rows step %row_step : i32 {
  %row         = tt.load %input_ptrs, %mask, %neg_inf :
  tensor<128x!tt.ptr<f32>>
  %row_max     = "tt.reduce"(%row) <{axis = 0}> ({
    ^bb0(%a: f32, %b: f32):
      %m = arith.maxnumf %a, %b : f32
      tt.reduce.return %m : f32
  }) : (tensor<128xf32>) -> f32
  %shifted     = arith.subf %row, %row_max_splat : tensor<128xf32>
  %numerator   = math.exp %shifted : tensor<128xf32>
  %denominator = "tt.reduce"(%numerator) <{axis = 0}> ({
    ^bb0(%a: f32, %b: f32):
      %s = arith.addf %a, %b : f32
      tt.reduce.return %s : f32
  }) : (tensor<128xf32>) -> f32
  %out         = arith.divf %numerator, %denominator_splat :
  tensor<128xf32>
  tt.store %output_ptrs, %out, %mask : tensor<128x!tt.ptr<f32>>
}
```

Interpreting this into Z3 and chasing the stored value's SSA def-use chain back to its terminals gives one closed-form formula for the whole block:

```
softmax_output = exp(load_row - maxnumf_row_minus_max) / addf_denominator
```

where `load_row`, `maxnumf_row_minus_max`, and `addf_denominator` are fresh, unconstrained Z3 symbols, and `exp` is the uninterpreted Z3 function standing in for `math.exp`.

This example demonstrates a real strength and two limitations of the Z3 interpreter at once:

– The elementwise arithmetic *around* the reductions (the subtraction and the final division) is captured exactly, so a candidate kernel that reorders or fuses these operations without changing their meaning still produces an identical formula.

– Both `tt.reduce` calls are opaque: the model proves the numerator is exactly "the row minus *some* value called the row max", but cannot verify that value, or the final sum, were actually computed as a correct max/sum over the row — a bug inside a reduction's own body (e.g. a wrong axis) is invisible to it.

  To address this limitation, the verifier treats each `tt.reduce` as an opaque operation and therefore separates the surrounding computation into symbolic regions. Within each region, Z3 provides the formal symbolic representation described above; the behavior of the reduction itself is represented by an opaque input. Reasoning about semantics that span the opaque boundary is then delegated to the LLM-based verification pipeline. This limits the amount of code for which the LLM must reason directly while avoiding an intractable expansion of the reduction into the SMT model.

– The surrounding `scf.for` is executed symbolically *once*. The formula above is therefore a per-row template, valid for a single grid-stride iteration. Thus, it cannot catch a bug that only manifests across iterations of the loop itself (e.g. wrong bounds, or a stride that causes rows to be skipped or revisited). Therefore, we rely on LLMs to reason across loop iterations.

This example illustrates the intended scope of the verifier: Z3 provides formal reasoning over supported symbolic regions, but does not attempt to model complete kernel semantics when operations fall outside that representation. The resulting decomposition concentrates formal verification where it is tractable and exposes the remaining semantic gaps explicitly for LLM-based reasoning.

D THE GLUON-TO-Z3 LIBRARY

The library models Gluon's explicit hardware constructs—threads, memory spaces, and the asynchronous copy, TMA (Tensor Memory Accelerator), and WGMMA (Warp Group Matrix Multiply-Accumulate) engines—as a bounded event graph: each hardware action becomes an event, ordered by two Z3 relations, *happens-before* (`HB`) and *program-order* (`PO`). A domain model (Table 5) defines the event vocabulary; a set of hardware-ordering *axioms* (Table 6) encodes real ordering rules as constraints on the graph; and a set of *theorems* (Table 7) checks a safety property by asking Z3 to satisfy its *negation*—UNSAT proves the property holds under every possible schedule, while SAT returns a concrete counter-example. Appendix D.1 works through exactly this mechanism end to end, on a realistic race.

Entity	Description
`Space`	Where an access happens: global memory, shared memory, registers, or (Blackwell) tensor memory.
`Proxy`	Which hardware engine issued an access: the generic load/store path, Ampere's asynchronous copy engine, the Hopper/Blackwell TMA engine, or the tensor core—PTX treats accesses issued by different proxies as unordered unless an explicit fence says otherwise.
`EventType`	The kind of bounded action an event represents: a load, a store, an atomic, a barrier arrival or wait, a proxy or memory-consistency fence, an MMA read or write, or a CTA-wide barrier.
`Event`	One bounded symbolic memory or synchronization action: which thread/CTA/cluster issued it, its space and proxy, its address (concrete or symbolic), its size, its position in program order, and its derived warp/lane.
`MBarrier`	Models an `mbarrier`: its address, how many arrivals it expects, and how many transaction bytes it expects (for TMA-signaled barriers).
`bank_of(addr)`	The shared-memory bank a (concrete or symbolic) address maps to.

Table 5: Domain model underlying the event graph. Hardware constants are also defined here: warp size, bank count and width, cache-line size, and each architecture's shared-memory capacity limit.

Axiom	Hardware rule encoded
Per-location sequential consistency `apply_sc_per_location()`	A sound simplification of full multi-copy-atomicity acyclicity: embeds happens-before into a single integer timeline, forcing it to be acyclic for every event pair.
CTA barrier `apply_cta_barrier(ids)`	Models `gl.barrier()`: for every same-CTA barrier pair, everything program-order-before the first barrier, on every participating thread, happens-before everything program-order-after the second.
Async-proxy fence `apply_fence_async_shared(id, cluster)`	Models `fence.proxy.async[.cluster]`, required before handing a shared buffer to TMA or the tensor core: prior generic-proxy shared writes on the fencing thread become happens-before-ordered relative to later async-proxy (TMA/tensor-core) accesses in the CTA (or cluster).
Mbarrier synchronization `apply_mbarrier_sync(barrier)`	Validates a barrier's arrival and transaction-byte accounting against its expected values. If short, adds *no* ordering edges and reports a deadlock (the wait blocks forever on real hardware); if satisfied, asserts happens-before from every arrival to every wait.
Manual ordering `assert_hb` / `assert_no_ordering`	Escape hatches to assert or deny a happens-before edge directly, for hardware relationships not covered by a mapper above, or for deliberately unsynchronized adversarial test cases.

Table 6: Axioms: hardware ordering rules encoded as constraints on the event graph. Each is applied explicitly by the code being verified, mirroring exactly which synchronization primitives that code actually issues.

Theorem	Property and mechanism
Async race freedom `check_async_race_freedom()`	For every event pair sharing an address and space with at least one write, asserts a race candidate: neither event happens-before the other. A same-thread generic-proxy write followed by an async-proxy (TMA/tensor-core) access additionally requires an intervening async-shared fence to count as visible—plain happens-before is not sufficient across a proxy boundary (Appendix D.1). SAT identifies a racing pair with a concrete model.
Bank-conflict freedom `check_bank_conflicts(ids)`	For same-warp, different-thread shared-memory accesses, two conflict iff they map to the same bank at different addresses (the same address is a free hardware broadcast, not a conflict). SAT over the disjunction of all pairwise conflict conditions.
Global coalescing `check_global_coalescing(ids)`	Introduces symbolic bounds on a warp's global accesses; SAT on either bound being misaligned to the cache-line size, or the span exceeding it, means the accesses cannot fit in one aligned, coalesced transaction.

Table 7: Theorems: each proves a safety property by attempting to satisfy its negation, returning `(is_safe, model)`.

D.1 Worked example: a race due to an omission of a fence

Gluon's explicit control over asynchronous engines is exactly where subtle hardware bugs hide, since the hardware imposes as little ordering as possible to maximize performance. This section walks through one such bug end-to-end (the code, the LLM's translation of it into the Gluon-to-Z3 library above, the counter-example Z3 returns, and how that counter-example becomes a concrete witness) using the exact tool output the harness produces.

1. The bug. In a warp-specialized, software-pipelined kernel, a producer warp finalizes a per-tile correction factor and writes it into a shared-memory scratch slot that a subsequent warpgroup MMA reads as one of its operands:

```
# l_smem is a shared-memory scratch buffer reused across pipeline stages.
l_smem[0] = l_i  # plain (generic-proxy) store

# BUG: no hopper.fence_async_shared() here. The tensor core reads l_smem
# through the *async* proxy, which is not guaranteed to observe a plain
# store without an intervening fence---even though, in program order,
# the store textually comes first.
acc = hopper.warpgroup_mma(a_smem, l_smem, acc)
```

Nothing here raises an error, and on some inputs and schedules the kernel may even produce correct output by chance—which is exactly why this class of bug is dangerous and why an empirical test suite alone is a weak defense against it (Section 3.3).

2. The translation. An LLM agent, working from Table 6/7 and the mapper signatures in Appendix D.2, maps the two operations above to events and asks for a race-freedom proof:

```
ctx = GluonVerificationContext()

# generic-proxy store: l_smem[0] = l_i
e_store = ctx.map_store(0, addr=0x100, space=Space.SHARED,
label="store_l_i")

# tensor-core read of the same address, via warpgroup_mma's l_smem
operand
reads, mma_write = ctx.map_warpgroup_mma(0, smem_addrs=0x100,
label="mma")

safe, model = ctx.check_async_race_freedom()
```

Note what is *absent*: no `map_fence_async_shared` call between the store and the MMA, because the kernel never issues one – the translation is a faithful mirror of the bug, not a fix for it.

3. The counter-example. Run through `run_z3` (Appendix B), this is the solver's actual output:

```
DATA RACE detected. Racing pairs consistent with this model:
  store_l_i(id=0, thread=0, STORE, SHARED@0x100, proxy=GENERIC)
    <-race-> mma.mma_read0(id=1, thread=0, MMA_READ, SHARED@0x100,
    proxy=TENSOR_CORE)
```

`check_async_race_freedom` is SAT: even though both events are issued by the same thread in program order, the proxy-visibility refinement in Table 7 means program order alone does not establish happens-before across a generic-to-async proxy boundary. The model names the exact two events, their addresses, and their proxies —enough on its own for the LLM to identify the missing `fence_async_shared()` call and where it belongs.

4. From counter-example to witness. A satisfying Z3 model is a proof artifact, not yet an empirical one: the verdict mechanism (Section 3.3) uses it, together with the qualitative failure it describes, to construct a concrete executable input that reproduces the same failure on real hardware. Here the model already pins down the exact racing address and event pair; in a kernel where that address is a symbolic function of, say, a pipeline-stage or tile index, Z3's satisfying assignment additionally fixes concrete values for those symbols, which translate directly into concrete launch parameters (which tile, which pipeline depth) that reproduce the identical address collision. Re-running the kernel under those parameters—optionally using the interactive debugger or `ncu`'s own race-detection instrumentation (Appendix B.2)—lets the LLM empirically confirm the race before treating a fix as validated, rather than trusting the symbolic proof alone.

D.2 Layout Algebra and Static Constraint Checks

The remaining part of the Gluon-to-Z3 library checks properties, for example those that depend on Gluon's *layouts* (the explicit mapping from a logical tensor coordinate to a physical register/lane/warp/address), rather than on event ordering.

`from_blocked_layout(size_per_thread, threads_per_warp, warps_per_cta, order, shape=None)`

Input the per-dimension extents of a `gl.BlockedLayout` at each hardware level (register, lane/thread, warp), and the dimension order (fastest-varying first).

Output a `LinearLayoutModel`: per-dimension basis vectors (register, lane, warp, block), combined by XOR over $GF(2)$ to recover a logical tensor coordinate—the same representation Triton's own compiler uses internally ("linear layouts").

`check_register_locality(layout, dim, num_warps=4)`

Input a `LinearLayoutModel`, a tensor dimension `dim`, and the CTA's warp count.

Output `(is_local, model)`. UNSAT proves `dim` is a pure function of the register index, independent of lane and warp—so a `gl.reduce` along that dimension needs zero cross-lane shuffles or shared-memory traffic.

`SwizzledSharedLayoutModel(vec, per_phase, max_phase, order)`

Input the four parameters of a `gl.SwizzledSharedLayout`.

Output a model of that layout's element-offset arithmetic (phase and XOR-swizzled column) as closed-form Z3 bit-vector expressions, for use by `check_shared_access_bank_conflicts` below.

`check_shared_access_bank_conflicts(addr_fn, num_lanes=32)`

Input a lane-to-address function, typically built from a `SwizzledSharedLayoutModel` above, and the number of lanes to check.

Output `(is_conflict_free, model)`. UNSAT proves that no two distinct lanes hit the same bank at different addresses, for all lane pairs at once.

`check_kernel_config(block_m, block_n, block_k, dtype_bytes=2, num_stages=1, arch="hopper", extra_smem_bytes=0)`

Input candidate block dimensions, element size, pipeline depth, target architecture, and any additional shared-memory the kernel reserves outside the pipelined buffers.

Output a static (solver-free) arithmetic check that the block dimensions are MMA/WGMMA-aligned and the per-stage shared-memory footprint fits the architecture's capacity—used to prune an autotuning search space before spending compile or GPU time on an infeasible configuration.

`check_pipeline_stage_isolation(num_stages, max_inflight=1, total_iters=100)`

Input the pipeline depth and how many stages may legitimately be in flight at once (a producer at iteration `iter_p` fills slot `iter_p % num_stages`; a consumer reads slot `(iter_p - lag) % num_stages` for any in-flight lag up to `max_inflight`).

Output `(is_isolated, model)`. SAT identifies a colliding iteration/lag pair—the producer overwrites a physical slot an in-flight consumer is still reading, which shows up as intermittent NaNs on real hardware.

`check_coalesced_stride(offset_fn, elem_bytes, base_addr=0, num_lanes=32)`

Input a lane-to-byte-offset function, the element size, the access's base address, and the number of lanes to check.

Output a check that adjacent lanes' offsets are exactly `elem_bytes` apart (contiguity) and that the base address is 16-byte-aligned (the `LDG.128` vectorized-load requirement); a violation of either indicates an uncoalesced or misaligned global access pattern.

D.3 Experiment Runtime and Total Cost

The following tables detail the wall-clock runtime and total token usage for each experiment-model pair.

Table 8: Optimization runs: total wall-clock time (hours).

Experiment	V4 Pro	V4 Flash	Muse 1.3	GPT-5.6 Sol
GPT-OSS	32	188	4	81
Qwen-3.5 MoE	21	17	5	2
Mamba-2	26	188	4	73
Cross-Entropy	11	36	4	72
Llama-4 RoPE	3	2	3	2
Neighborhood-Attention	18	10	3	2
Total	**111**	**441**	**23**	**232**

Table 9: Optimization runs: total tokens.

Experiment	V4 Pro	V4 Flash	Muse 1.3	GPT-5.6 Sol
GPT-OSS	50.0 M	121.1 M	21.6 M	11.8 M
Qwen-3.5 MoE	65.4 M	68.3 M	24.6 M	6.5 M
Mamba-2	86.1 M	146.0 M	26.2 M	10.8 M
Cross-Entropy	68.6 M	68.8 M	23.5 M	5.5 M
Llama-4 RoPE	20.8 M	23.9 M	9.7 M	6.9 M
Neighborhood-Attention	69.4 M	59.0 M	22.4 M	10.1 M
Total	**360.3 M**	**487.1 M**	**128.0 M**	**51.6 M**

E EXPERIMENT DATA

All experiments were conducted on an NVIDIA DGX Spark with a GB10 chip. In this section we give more details regarding the experiment setup and data collected.

E.1 KERNELS UNDER OPTIMIZATION

Table 10 lists, for each experiment, the Triton kernels Glueball was tasked with optimizing.

Experiment	#	Kernels
GPT-OSS	1	`_attn_fwd`
Qwen-3.5 MoE	2	`_fused_up_proj_swiglu_kernel` `_fused_down_proj_kernel`

Mamba-2	7	`_chunk_cumsum_fwd_kernel`
		`_chunk_state_fwd_kernel`
		`_state_passing_fwd_kernel`
		`_bmm_chunk_fwd_kernel`
		`_chunk_scan_fwd_kernel`
		`_layer_norm_fwd_1pass_kernel`
		`_selective_scan_update_kernel`

Cross-Entropy	1	`liger_cross_entropy_kernel`
Llama-4 RoPE	1	`_llama4_rope_kernel`

Neighborhood-Attention	5	`_fused_neighborhood_attention_qk_kernel`
		`_softmax_single_block_forward_kernel`
		`_softmax_multi_block_forward_kernel`
		`_neighborhood_mask_kernel`
		`_fused_neighborhood_attention_av_kernel`

Table 10: Kernels put through the Glueball harness.

E.2 OPTIMIZATION PERFORMANCE ACROSS ROUNDS

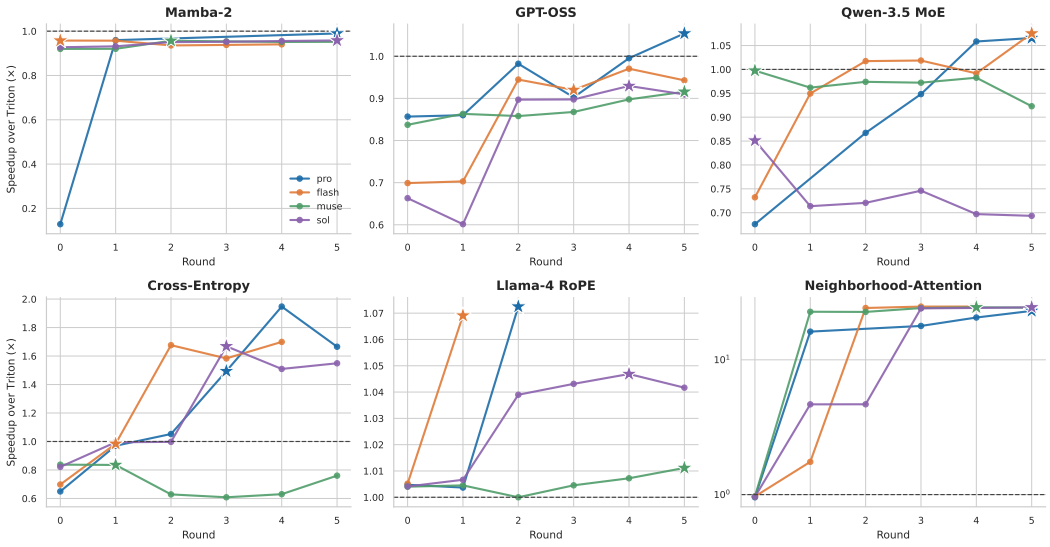


Figure 7: **End-to-End performance across rounds.** Star indicates the best round. Each trajectory may end before reaching the last round if the Proposal Agent fails to propose new optimizations. Each point corresponds to the median performance across a range of shapes, detailed in Appendix E.3.

Shape	Triton (ms)	V4 Pro	V4 Flash	Muse 1.3	5.6 Sol
b=8 H=16 L=2048 d=96 k=7 dil=1	39.079	39.346×	43.322×	39.554×	43.125×
b=2 H=16 L=4096 d=128 k=7 dil=1	40.969	63.005×	66.577×	64.322×	66.570×
b=8 H=16 L=2048 d=128 k=7 dil=1	44.380	35.322×	36.981×	35.968×	37.491×
b=2 H=8 L=8192 d=64 k=7 dil=1	54.388	159.499×	162.956×	158.562×	164.438×
b=8 H=8 L=4096 d=64 k=7 dil=1	56.969	89.525×	91.213×	90.178×	91.971×
b=2 H=8 L=8192 d=96 k=7 dil=1	72.072	139.079×	152.736×	149.165×	151.534×
b=2 H=8 L=8192 d=128 k=7 dil=1	78.620	120.255×	126.859×	124.683×	127.319×
b=8 H=8 L=4096 d=96 k=7 dil=1	78.986	79.610×	87.550×	81.616×	87.553×
b=8 H=8 L=4096 d=128 k=7 dil=1	88.643	70.785×	74.052×	71.547×	74.819×
b=2 H=16 L=8192 d=64 k=7 dil=1	111.790	174.672×	179.556×	175.828×	181.045×
b=8 H=16 L=4096 d=64 k=7 dil=1	116.437	94.516×	97.104×	93.170×	97.520×
b=2 H=16 L=8192 d=96 k=7 dil=1	149.874	153.577×	166.699×	159.811×	166.131×
b=8 H=16 L=4096 d=96 k=7 dil=1	155.451	80.367×	88.256×	82.104×	88.209×
b=2 H=16 L=8192 d=128 k=7 dil=1	165.027	131.510×	137.405×	135.449×	139.287×
b=8 H=16 L=4096 d=128 k=7 dil=1	175.604	71.039×	74.237×	72.056×	75.982×

E.4 Correctness Suite

We detail any edits and injected bugs in our Generic and Synchronization correctness suites.

E.4.1 Generic Correctness Suite

Table 17: Generic Correctness Suite: semantics-preserving changes.

#	Kernel	Change made
1	_fused_add_rms_norm_forward_kernel	Identity
2	_fused_add_rms_norm_forward_kernel	Extract an expression into a helper
3	_fused_add_rms_norm_forward_kernel	Insert a comment
4	_fused_add_rms_norm_forward_kernel	Rename a local variable
5	_geglu_tanh_forward_kernel	Identity
6	_geglu_tanh_forward_kernel	Extract an expression into a helper
7	_geglu_tanh_forward_kernel	Insert a comment
8	_geglu_tanh_forward_kernel	Reassociate a product
9	_geglu_tanh_forward_kernel	Rename a local variable
10	_kldiv_kernel_forward	Identity
11	_kldiv_kernel_forward	Extract an expression into a helper
12	_layer_norm_forward_kernel	Identity
13	_layer_norm_forward_kernel	Extract an expression into a helper

continued on next page

#	Kernel	Change made
14	_layer_norm_forward_kernel	Insert a comment
15	_layer_norm_forward_kernel	Rename a local variable
16	_rms_norm_forward_kernel	Identity
17	_rms_norm_forward_kernel	Extract an expression into a helper
18	_rms_norm_forward_kernel	Insert a comment
19	_rms_norm_forward_kernel	Rename a local variable
20	_softmax_single_block_forward_kernel	Identity
21	_softmax_single_block_forward_kernel	Extract an expression into a helper
22	_softmax_single_block_forward_kernel	Insert a comment
23	_swiglu_forward_kernel	Identity
24	_swiglu_forward_kernel	Extract an expression into a helper
25	_swiglu_forward_kernel	Insert a comment
26	_swiglu_forward_kernel	Rename a local variable

Table 18: Generic Correctness Suite: injected bugs.

#	Kernel	Bug added
1	_fused_add_rms_norm_forward_kernel	Bounds mask < becomes <=
2	_fused_add_rms_norm_forward_kernel	Drop the row-stride term
3	_fused_add_rms_norm_forward_kernel	Drop the rsqrt epsilon guard
4	_fused_add_rms_norm_forward_kernel	Reduce along the wrong axis
5	_geglu_tanh_forward_kernel	Bounds mask < becomes <=
6	_geglu_tanh_forward_kernel	Drop the row-stride term
7	_geglu_tanh_forward_kernel	Negate the activation
8	_kldiv_kernel_forward	Accumulate in a narrower dtype
9	_kldiv_kernel_forward	Bounds mask < becomes <=
10	_kldiv_kernel_forward	Reduce along the wrong axis
11	_layer_norm_forward_kernel	Accumulate in a narrower dtype
12	_layer_norm_forward_kernel	Bounds mask < becomes <=
13	_layer_norm_forward_kernel	Drop the rsqrt epsilon guard
14	_layer_norm_forward_kernel	Reduce along the wrong axis
15	_rms_norm_forward_kernel	Bounds mask < becomes <=
16	_rms_norm_forward_kernel	Drop the rsqrt epsilon guard
17	_rms_norm_forward_kernel	Reduce along the wrong axis
18	_softmax_single_block_forward_kernel	Accumulate in a narrower dtype
19	_softmax_single_block_forward_kernel	Bounds mask < becomes <=
20	_softmax_single_block_forward_kernel	Drop the softmax max subtraction
21	_softmax_single_block_forward_kernel	Masked-load fill $-\infty$ becomes 0
22	_softmax_single_block_forward_kernel	Reduce along the wrong axis
23	_swiglu_forward_kernel	Bounds mask < becomes <=
24	_swiglu_forward_kernel	Drop the row-stride term

E.4.2 SYNCHRONIZATION CORRECTNESS SUITE

Table 19: Synchronization Correctness Suite: semantics-preserving changes.

#	Kernel	Change made
1	`elementwise_add_cpasync_kernel`	Identity
2	`elementwise_add_kernel`	Identity
3	`elementwise_add_pipelined_kernel`	Identity
4	`elementwise_add_tma_kernel`	Hoist `read_index % num_buffers` into a local
5	`elementwise_add_tma_kernel`	Identity
6	`elementwise_add_warp_specialized_kernel`	Identity
7	`memcpy_1d_cpasync_kernel`	Identity
8	`memcpy_1d_kernel`	Identity
9	`memcpy_1d_tma_kernel`	Identity
10	`memcpy_2d_inout_kernel`	Identity (separate in/out descriptors)
11	`memcpy_2d_kernel`	Identity
12	`memcpy_kernel`	Identity

Table 20: Synchronization Correctness Suite: injected bugs.

#	Kernel	Bug added
1	`elementwise_add_pipelined_kernel`	Drain waits `num_buffers-1-i`, never reaching 0
2	`elementwise_add_pipelined_kernel`	Steady state waits `wait_group(num_buffers)`
3	`elementwise_add_tma_kernel`	`fence_async_shared()` moved before the shared store
4	`elementwise_add_warp_specialized_kernel`	`fence_async_shared()` moved after `mbarrier.arrive`
5	`elementwise_add_warp_specialized_kernel`	Shared loads moved after `arrive(load_empty_bar)`
6	`memcpy_1d_cpasync_kernel`	`cp.wait_group(0)` becomes `gl.barrier()`
7	`memcpy_1d_kernel`	Bounds mask replaced by a tautology
8	`memcpy_1d_tma_kernel`	`mbarrier.expect` halved to `nbytes // 2`
9	`memcpy_2d_kernel`	Output offsets use the input strides
10	`memcpy_kernel`	Drop the tail clamp from the loop bound

F Complete Agent System Prompts

This section lists the system prompt for each agent role. Some infrastructure and prompt fragments are shared by multiple agents (indicated via **"composed with"**, and are listed under the Shared Infrastructure and Shared Fragments subsections respectively.

Contents of this section:

Optimizer Pipeline
Translation, Profiling, Proposal, Model Proposal, Optimization, Benchmark Setup, Launch Code Generator

Correctness Pipeline
Splitter, Mapper, Equivalence, Verdict

Shared Infrastructure
Documentation Assistant, Tool-Output Summarizer

Shared Fragments
Section F.4

F.1 Optimizer Pipeline

F.1.1 Translation

Function: Translates a reference Triton kernel into a correct-but-unoptimized Gluon kernel. It is used to seed the Gluon optimization pipeline.

Composed with: `Complex-kernel development note`, `Reference-algorithm note (optimizer)`, `Gluon translator tool (seed variant)`, `Shell access block`.

Full prompt (8,559 characters):

```
You are a GPU kernel engineer who specialises in translating Triton
DSL kernels
into semantically equivalent Gluon kernels.

Gluon is Triton's lower-level GPU programming model targeting Hopper
and Blackwell
GPUs.  Key differences from the Triton DSL:


## Structure
- A Gluon file normally has MULTIPLE `@gluon.jit` functions:
  - One or more helper "warpgroup" routines (`_producer_fn`,
  `_consumer_fn`, etc.)
  - A main launcher function (the last `@gluon.jit` entry)
  dispatching warpgroups.
- Warp specialization is explicit: producer warps run TMA loads;
consumer warps
  run WGMMA math.  You must partition warps manually.


## Core primitives
- **TMA async loads** | `tma.async_load(desc, smem_buf, phase, ...)`
- **WGMMA** | `warpgroup_mma(acc, a_smem, b_desc)` /
`warpgroup_mma_wait(n)`
- **MBarrier** | `mbarrier.init(n_warps)` / `.arrive()` /
`.wait(phase)`
- **Tensor Memory (TMEM)** | `gl.tmem_alloc(shape, dtype)` for
accumulators
- **Fences** | `gl.fence_async_shared()` before arriving on shared
data
```